

Model Assessment and Validation

Goals and assigned reading

The topics in this chapter are:

Bias-variance trade off in model complexity

Uses of training and test data

Cross-validation

Performance metrics for model selection

Assigned Reading: Read Chapter 7 of *The Elements of Statistical Learning*. This chapter gives an overview of key concepts of model assessment, including the bias-variance tradeoff, training and test data, cross-validation, and performance metrics for models. I am assuming that you now know the material of Chapter 3 on linear regression. We have skipped Chapters 4-6 for now and will return to those.

Note that Chapter 5 of *An Introduction to Statistical Learning* is also useful.

Introduction: the bias-variance tradeoff

In fitting models to data, we have a trade off between **bias** and **variance**. The **bias** in a model is the error that results because the model is not a true representation of the underlying relationship. For example, if we use a linear model for data where the underlying relationship between response and predictors is not truly linear. The **variance** in a model fit refers to the amount by which the fitted model would change if estimated it using a different data set.

Ideally, we would like a model to have low bias and low variance. However, in reality, decreasing one type of error often causes the other type of error to increase. More complex models can fit the data more closely and (possibly) better capture the underlying relationships in the data. However, more complex models may fit to random variation and not true underlying relationships. This is called **overfitting**. An overfitted model will fit the current data well, but will generalize poorly to another data set.

Measuring how well a model fits the data

Suppose that we have data consisting of a response variable Y and predictors $X = X_1, X_2, \dots, X_p$. We assume that there is a relationship between X and Y of form

$$Y = f(X) + \epsilon$$

Where f is some unknown function of X and ϵ is random error term that is independent of X and has mean zero.

Our goal is to fit a model $\hat{f}(x)$ that represents $f(x)$ as well as possible.

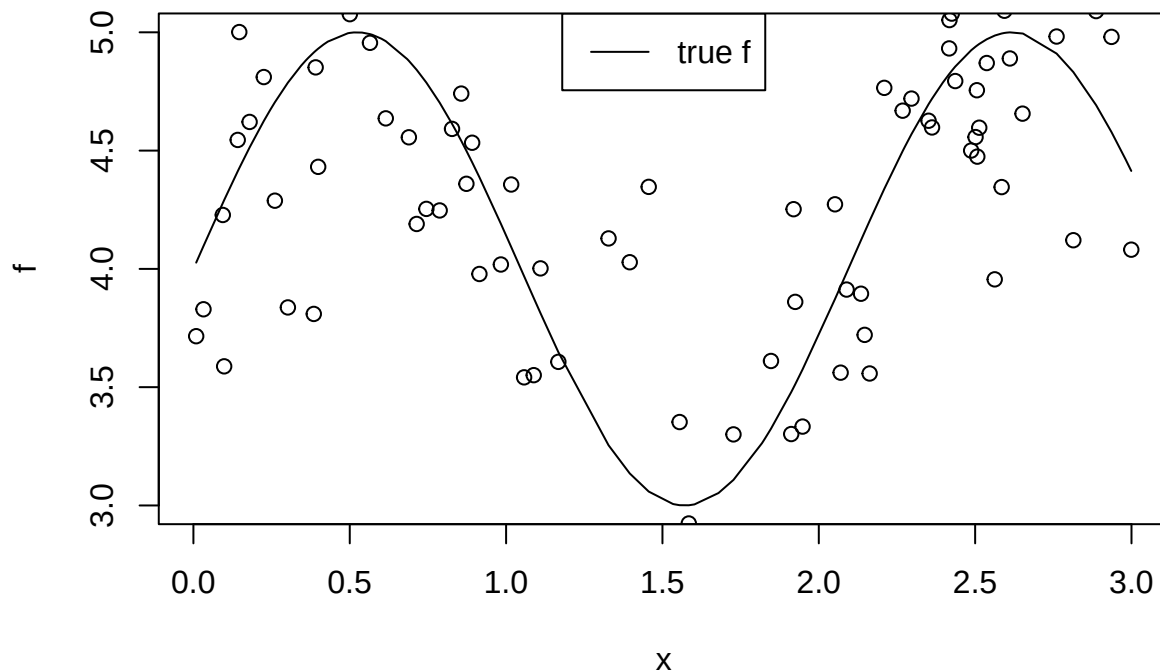
Our most common metric of how well a model fits the data is the mean squared error (MSE):

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{f}(x_i))^2$$

Example 1: complex X-Y relationship, moderate noise

Consider the data below that is simulated from a sine curve plus noise:

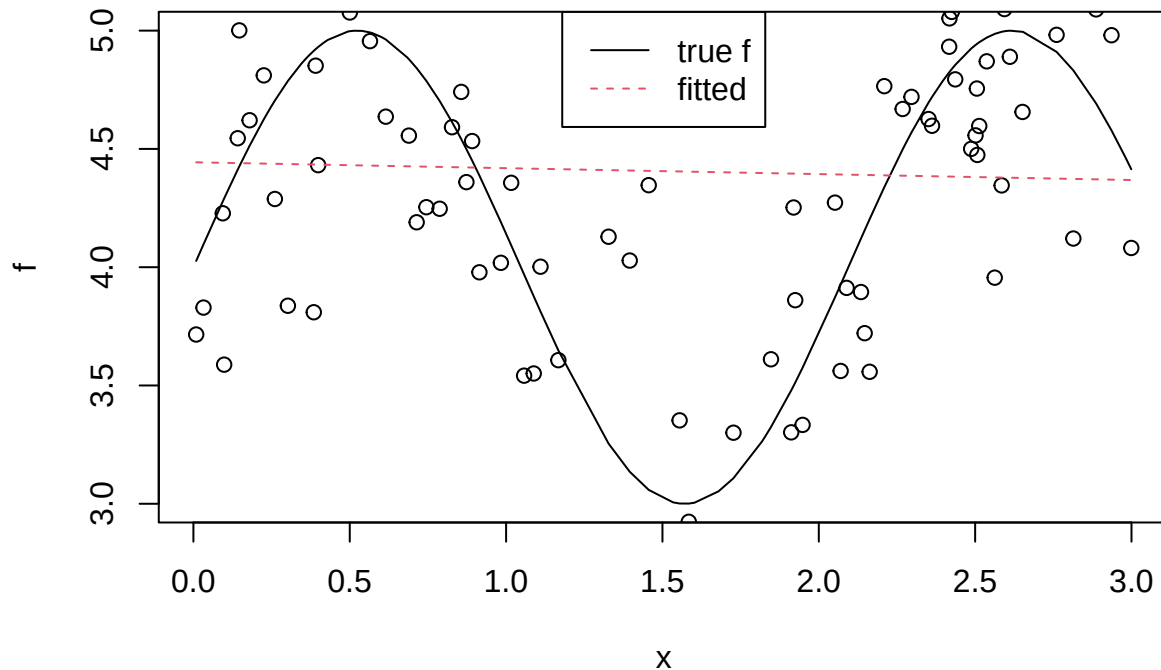
```
set.seed(10202) #setting the seed make the simulated be the same every time.  
               #okay for demonstration purposes, but not simulations for which  
               #you will use the results  
  
n=100  
x=sort(runif(n,0,3))  
f=4+sin(3*x)  
y=f+rnorm(n,sd=0.5)  
  
plot(x,f,type="l")  
points(x,y)  
legend("top","true f",lty=1)
```



```
#plot(x,y)
```

Suppose that we fit a straight line to this data:

```
linmod=lm(y~x)  
plot(x,f,type="l")  
points(x,y)  
lines(x,fitted(linmod),col=2,lty=2)  
legend("top",c("true f","fitted"),col=1:2,lty=1:2)
```



```
#MSE
mean((y-fitted(linmod))^2)
```

```
## [1] 0.7317105
```

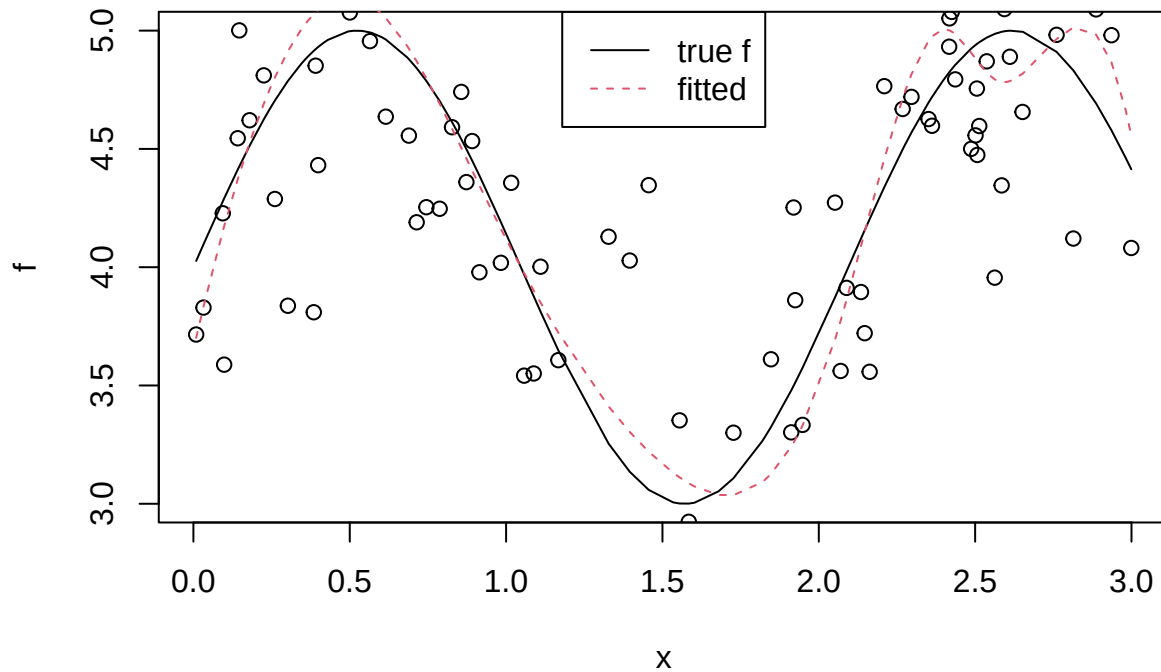
A straight line is a poor fit. This produces an MSE of 0.73. The high MSE is due to a bias in the fit - there is high error because the model is a poor representation of the underlying data.

Next, we use a more complex model. We use a spline, which breaks up the x-axis into intervals and fits a straight line in each one. The more intervals, the more the fit can adapt to the data. We will discuss splines in detail later in the semester.

```
library(splines)

splineMod=lm(y~bs(x,df=10))

plot(x,f,type="l")
points(x,y)
lines(x,fitted(splineMod),col=2,lty=2)
legend("top",c("true f","fitted"),col=1:2,lty=1:2)
```



We see that the spline gives a model fit much closer to the true relationship between X and Y . The MSE is much smaller:

```
mean((y-fitted(splinemod))^2)
```

```
## [1] 0.2575935
```

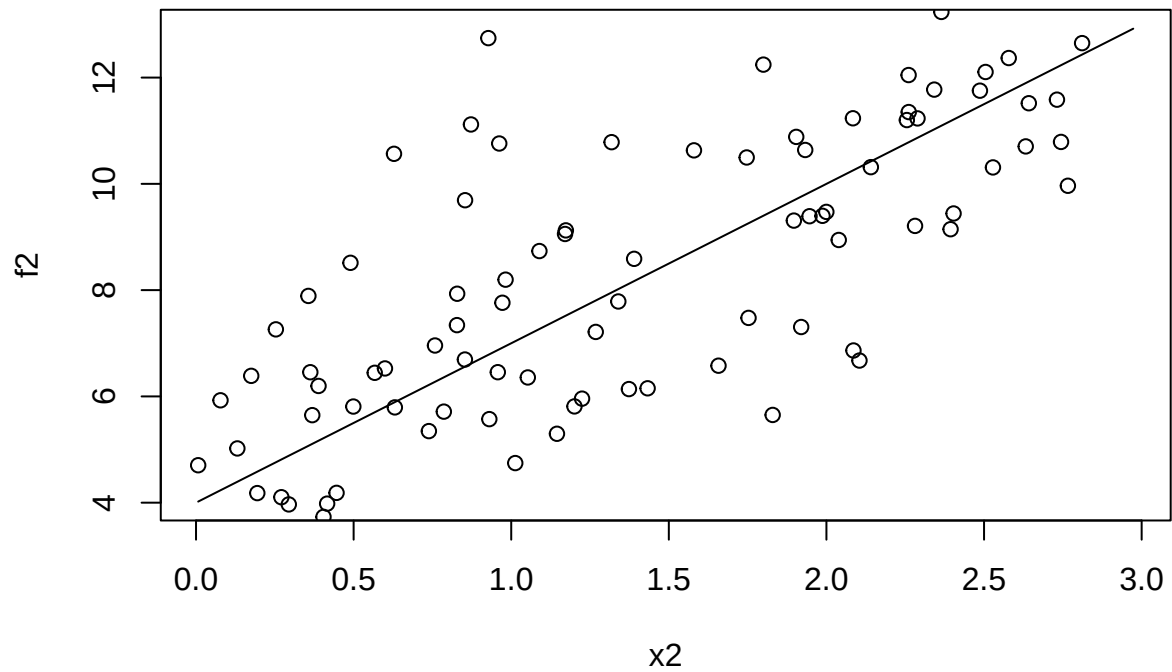
Although not perfect, the spline is a much better representation of the true X and Y relationship and most of the MSE is due to variance - that is, random variation of the data around f .

Example 2: Linear relationship with higher noise

Let's consider a second example where the underlying relationship is linear, but there is more noise:

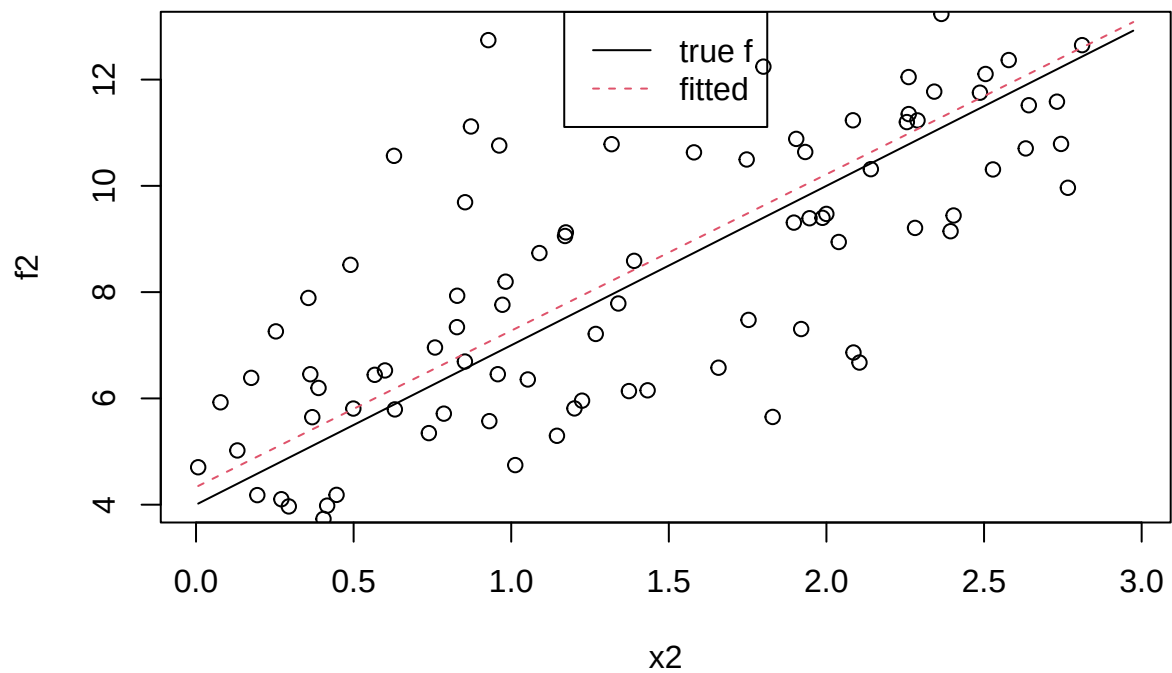
```
n=100
x2=sort(runif(n,0,3))
f2=4+3*x2
y2=f2+rnorm(n,sd=2)

plot(x2,f2,type="l")
points(x2,y2)
```



Fit a straight line:

```
linmod2=lm(y2~x2)
plot(x2,f2,type="l")
points(x2,y2)
lines(x2,fitted(linmod2),col=2,lty=2)
legend("top",c("true f", "fitted"),col=1:2,lty=1:2)
```



In this case, the fitted line is very close to the true relationship between X and Y . There is still higher MSE than the previous example:

```
mean((y2-fitted(linmod2))^2)
```

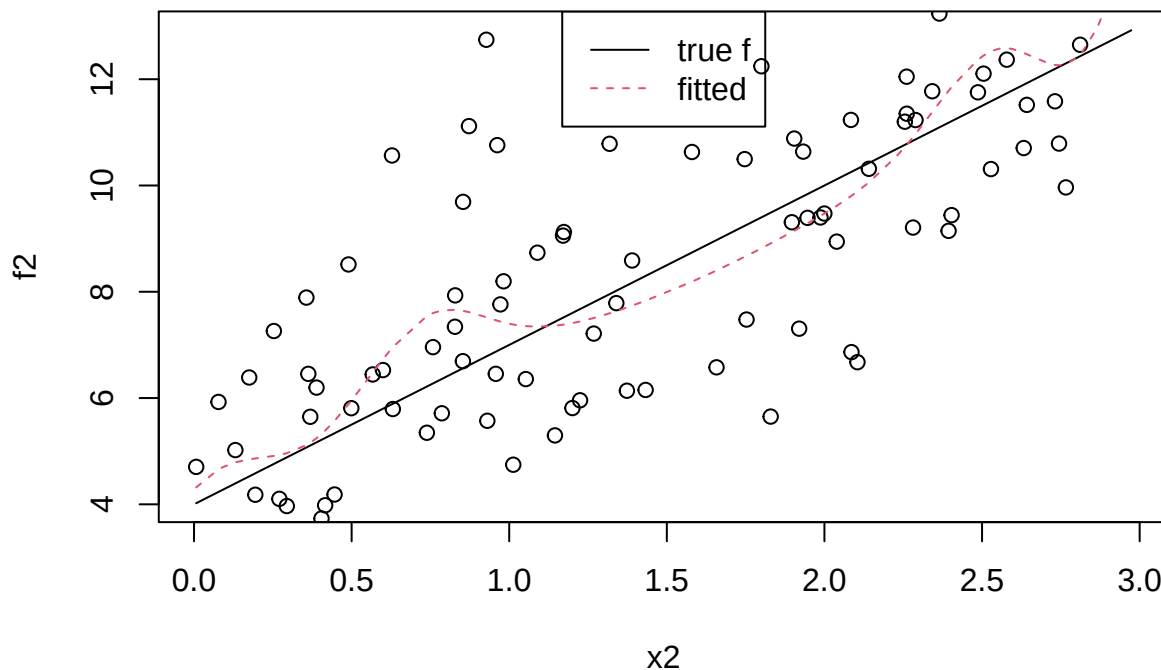
```
## [1] 3.980044
```

This is because this data has higher random variation. What happens if we fit a spline?

```
library(splines)

splineMod2=lm(y2~bs(x2,df=10))

plot(x2,f2,type="l")
points(x2,y2)
lines(x2,fitted(splineMod2),col=2,lty=2)
legend("top",c("true f", "fitted"),col=1:2,lty=1:2)
```



```
mean((y2-fitted(splineMod2))^2)
```

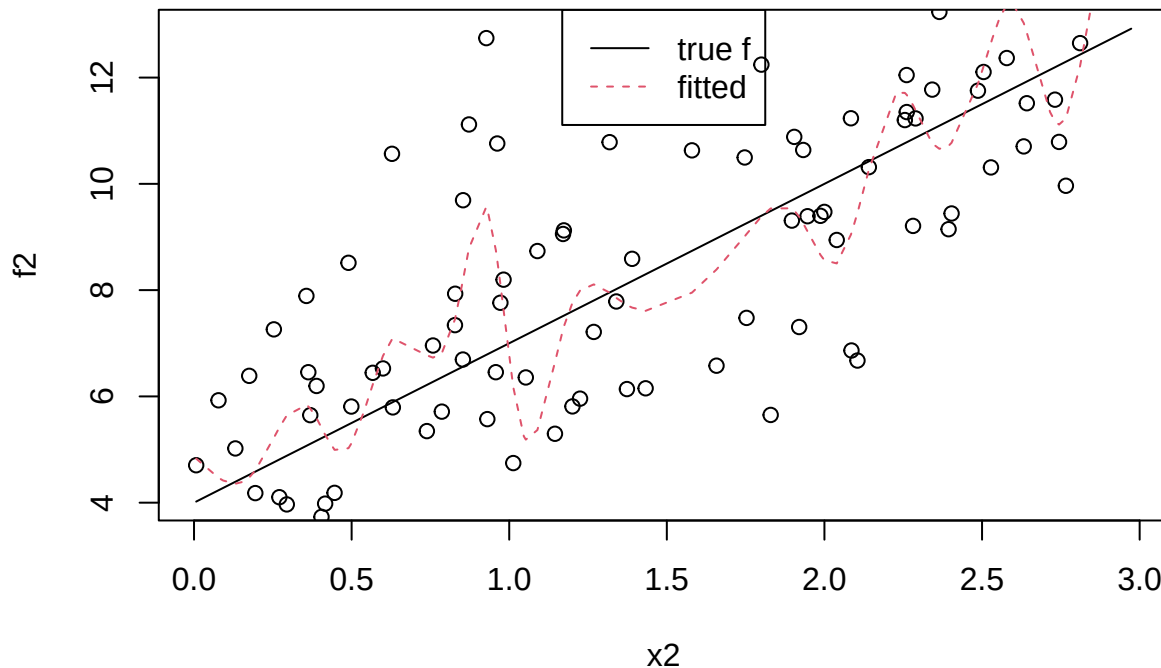
```
## [1] 3.631914
```

We get lower MSE. Let's fit another spline that breaks up X into more intervals (determined by the `df` argument). This gives a more complex model fit that will fit the data more closely and reduce MSE:

```
library(splines)

splineMod3=lm(y2~bs(x2,df=20))

plot(x2,f2,type="l")
points(x2,y2)
lines(x2,fitted(splineMod3),col=2,lty=2)
legend("top",c("true f", "fitted"),col=1:2,lty=1:2)
```



```
mean((y2-fitted(splinemod3))^2)
```

```
## [1] 3.005996
```

This model produces less MSE than the straight line fit, but is this a good thing? We see from the plot that the spline is fitting to random variation in the data, not the true underlying relationship.

Predicting new data

In prediction, our goal is not merely to predict values in one data set. We want to use one data set to train the model and then use the model to predict future values of the response variable given the predictors. This is harder. To see why, use the models from the above example to predict Y for a new data set:

```
#generate new data from the same model:
n=100
x3=sort(runif(n,0,3))
f3=4+3*x3
y3=f3+rnorm(n,sd=2)

#predictions for the new Y values from the models trained above:
predy_lm=predict(linmod2,newdata=data.frame(x3))
predy_spline2=predict(splinemod2,newdata=bs(x3,df=10))
predy_spline3=predict(splinemod3,newdata=bs(x3,df=20))

#MSE for the new predictions:
mean((y3-predy_lm)^2)
```

```
## [1] 4.727087
```

```
mean((y3-predy_spline2)^2)
```

```
## [1] 4.916284
```

```
mean((y3-predy_spline3)^2)
```

```
## [1] 6.227005
```

With the new data, the MSE is lowest for the linear model, second lowest for the spline with 10 degrees of freedom, and highest for the model with 20 degrees of freedom. This is because the more complex models are overfitting the data - that is fitting to random variation and not the true relationship. Future data won't have the same randomness as the training data. If the training data fits to noise, it will make poorer predictions for future data.

The problem is that in a real data situation, we won't know what the true underlying relationship is between X and Y and so we won't know if we are fitting noise or fitting the true relationship.

This trade off - making the model complex enough to fit the true relationship but not complex enough that it fits noise - is one of the central challenges of prediction

Exercise #1 Conduct a simulation to compare the distribution of MSE on new data versus the original for the above data generating model when fitted by linear model and splines with 10 and 20 df.

```
#code code completed in class
```

Training and Test Data

One solution to this problem is to have *training* and *testing* data:

Training data: data used for constructing a model. The prediction error (**training error**) calculated from training data decreases with model complexity (decrease in bias). However, a model with zero training error overfits the training data and typically generalizes poorly (increase in variance).

Testing data: independent data set used for testing the fitted model. Prediction capability on this set is called **test error**. We choose and evaluate models based on test error. Training error is typically less than the test error since for the training error the same data are used to fit the method and assess its error (double dipping!)

Validation data: Many methods that we will discuss have a tuning parameter of some sort that determines the complexity of the model fit. In such cases, a validation data set is used to fit the value of the tuning parameter.

If the data is large enough, we can randomly divide it (or generate data for simulation) into a training data set and a testing data set. One can also divide into three: training, validation (tuning), and test if there are tuning parameters in a model.

Example with Auto data

The ISLR library contains the data sets used in **An Introduction to Statistical Learning with Applications in R**.

We will use the **Auto** data set. This contains the gas mileage, horsepower, and other information for 392 different car models.


```
library(ISLR)
```

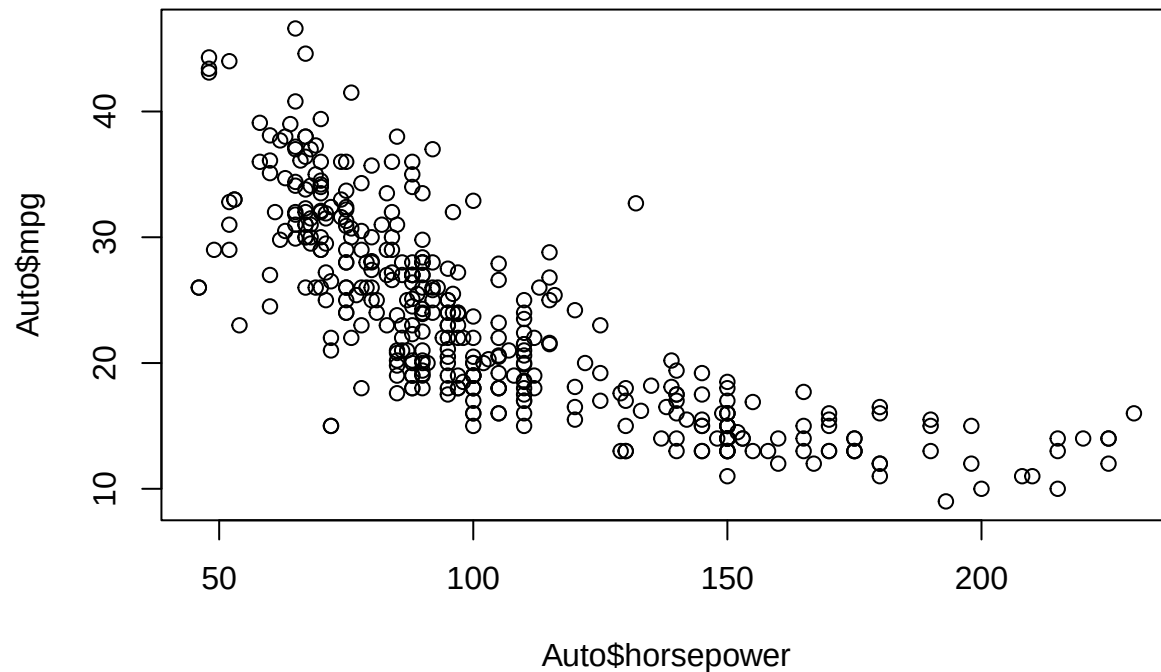
```
## Warning: package 'ISLR' was built under R version 4.5.2
```

```
head(Auto)
```

```
##   mpg cylinders displacement horsepower weight acceleration year origin
## 1  18         8         307         130   3504          12.0    70      1
## 2  15         8         350         165   3693          11.5    70      1
## 3  18         8         318         150   3436          11.0    70      1
## 4  16         8         304         150   3433          12.0    70      1
## 5  17         8         302         140   3449          10.5    70      1
## 6  15         8         429         198   4341          10.0    70      1
##                                name
## 1 chevrolet chevelle malibu
## 2      buick skylark 320
## 3    plymouth satellite
## 4      amc rebel sst
## 5      ford torino
## 6    ford galaxie 500
```

```
?Auto
```

```
plot(Auto$horsepower, Auto$mpg)
```



We will randomly split the data into three (almost) equal parts: a training data set, a validation data set, and a test data set.

```
#library(ISLR)
set.seed(3)
sample_shuffle=sample(392) #randomly shuffle the sample
#choose 33% of the data for each part:
train=sample_shuffle[1:131]
validate=sample_shuffle[132:262]
test=sample_shuffle[263:392]
```

The looks like it has some curvature and therefore it may make sense to fit a higher degree polynomial. However, we don't know what degree. This is an example of the model complexity trade off. A higher degree may fit the underlying relationship better or it may fit to noise.

We will follow three steps:

1. Using the training data, fit the data with models ranging from degree 1 (linear) to degree 6.
2. Use the validation data to compare the MSE of the fitted models. We will take the model with the lowest MSE as the best model.
3. We will then evaluated that model on the test data.

Fit the models with training data:

```
#fit the model to the training data only

attach(Auto)

#linear model:
lm.fit=lm(mpg~horsepower,data=Auto,subset=train)

#Repeat with a quadratic model:
lm.fit2=lm(mpg~poly(horsepower,2),data=Auto,subset=train) # quadratic

#Repeat with a cubic model:
lm.fit3=lm(mpg~poly(horsepower,3),data=Auto,subset=train) #cubic

#Repeat with a degree 4 model:
lm.fit4=lm(mpg~poly(horsepower,4),data=Auto,subset=train) #4th power

#Repeat with a degree 5 model:
lm.fit5=lm(mpg~poly(horsepower,5),data=Auto,subset=train) #5th power

#Repeat with a degree 6 model:
lm.fit6=lm(mpg~poly(horsepower,6),data=Auto,subset=train) #6th power

#Repeat with a degree 7 model:
lm.fit7=lm(mpg~poly(horsepower,7),data=Auto,subset=train) #7th power

#Repeat with a degree 8 model:
lm.fit8=lm(mpg~poly(horsepower,8),data=Auto,subset=train) #8th power
```

Apply the fitted models to the validation data and calculate the MSE:

```
mean((mpg-predict(lm.fit,Auto))[validate]^2)
```

```
## [1] 24.75917
```

```
mean((mpg~predict(lm.fit2,Auto))[validate]^2) #calculate squared prediction error for validation data
```

```
## [1] 20.93495
```

```
mean((mpg~predict(lm.fit3,Auto))[validate]^2)
```

```
## [1] 20.98526
```

```
mean((mpg~predict(lm.fit4,Auto))[validate]^2)
```

```
## [1] 21.13211
```

```
mean((mpg~predict(lm.fit5,Auto))[validate]^2)
```

```
## [1] 20.6503
```

```
mean((mpg~predict(lm.fit6,Auto))[validate]^2)
```

```
## [1] 20.00785
```

```
mean((mpg~predict(lm.fit7,Auto))[validate]^2)
```

```
## [1] 19.45893
```

```
mean((mpg~predict(lm.fit8,Auto))[validate]^2)
```

```
## [1] 19.53152
```

The MSE in the validation data is the smallest for the degree 7 model. Thus we take this as the final model.

Evaluate the final model on the test data

```
mean((mpg~predict(lm.fit7,Auto))[test]^2)
```

```
## [1] 18.43096
```

Compare to the MSE on the training data:

```
mean((mpg~predict(lm.fit7,Auto))[train]^2)
```

```
## [1] 16.70732
```

We see that the MSE on the training data is a good deal lower. This is as expected because the model will be fit to random features of the training data.

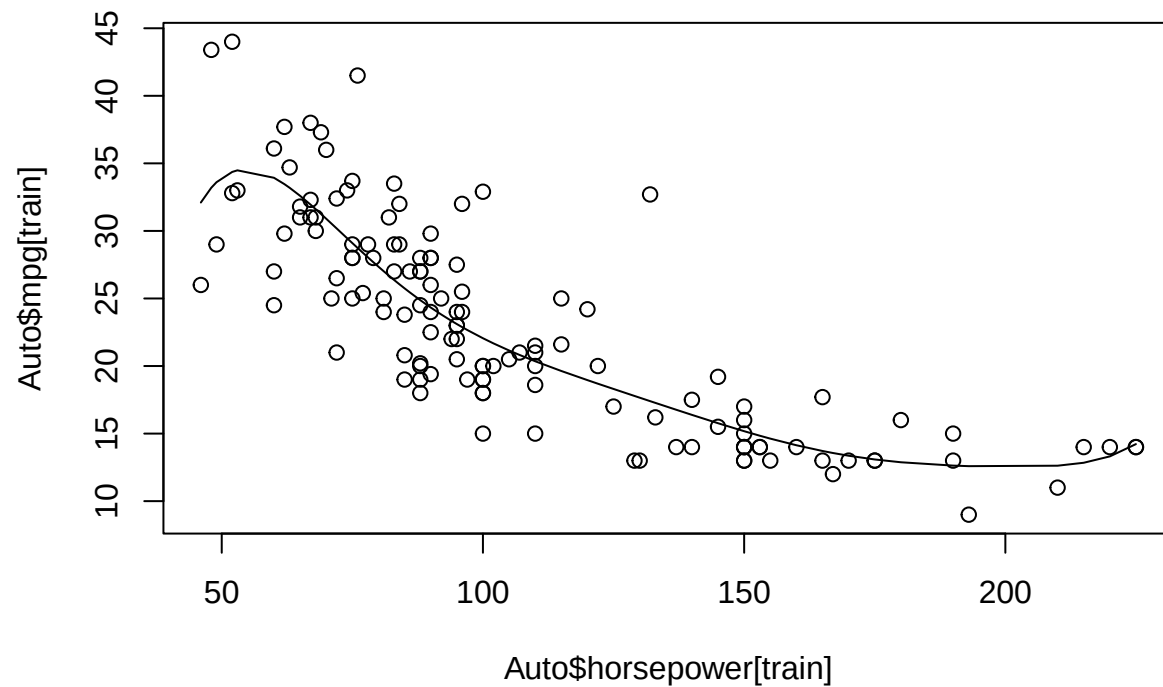
Plot the fitted model with the training data and test data:

```

pred_train=predict(lm.fit7,newdata=Auto[train,])
ord_train=order(Auto$horsepower[train])
plot(Auto$horsepower[train],Auto$mpg[train], main="fitted model with training data")
lines(Auto$horsepower[train][ord_train], pred_train[ord_train])

```

fitted model with training data

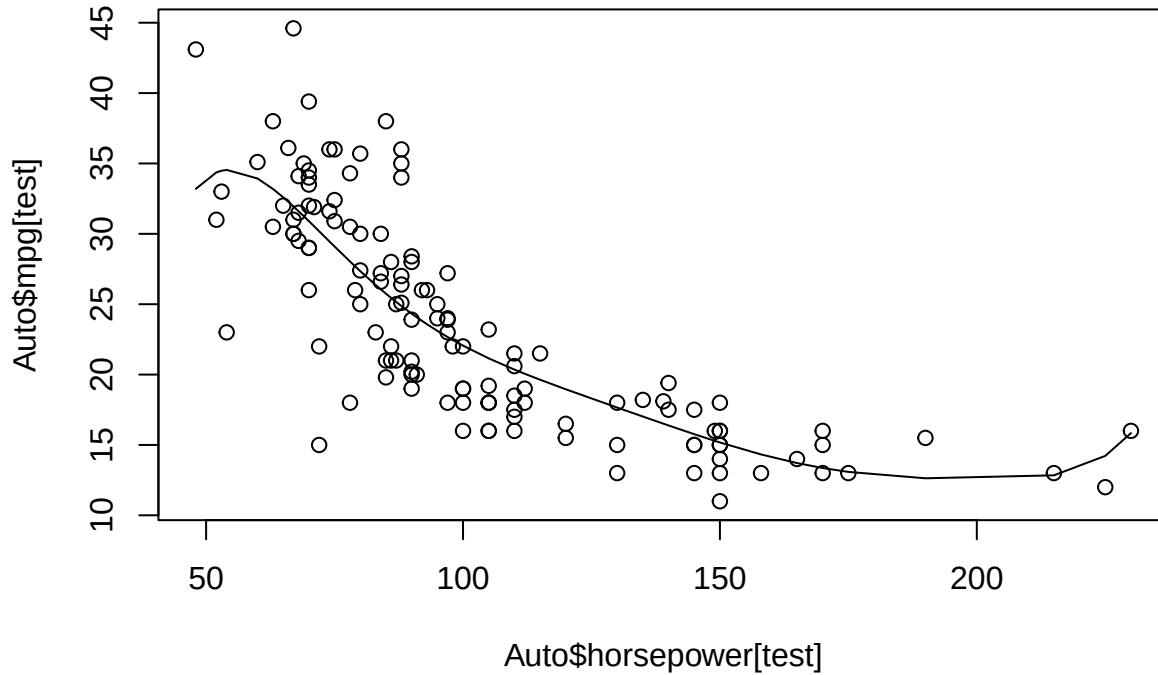


```

pred_test=predict(lm.fit7,newdata=Auto[test,])
ord_test=order(Auto$horsepower[test])
plot(Auto$horsepower[test],Auto$mpg[test], main="fitted model with test data")
lines(Auto$horsepower[test][ord_test], pred_test[ord_test])

```

fitted model with test data



It is subtle, but the fitted model appears to fit the training data better than the test data.

Alternatives to Test and Training Data

If the data is insufficient to divide into multiple parts, there are two alternative methods to approximate the validation step.

- Nonparametric methods: efficient sample reuse - cross-validation (CV), bootstrap
- Model-based methods: covariance penalties such as Cp, AIC, BIC, etc.

Leave-One-Out Cross-Validation (LOOCV)

Cross validation estimates the test error by holding out a subset of the training observations from the fitting process, and then applying the statistical learning method to those held out observations.

In Leave-One-Out Cross Validation, a single observation (e.g. (x_1, y_1)) is used for the validation set, and the remaining observations make up the training set. The statistical learning method is fit on the $n - 1$ training observations (e.g. $\{(x_2, y_2), \dots, (x_n, y_n)\}$), and a prediction $\hat{y}_1^{(1)}$ is made for the excluded observation, using its value x_1 . Since (x_1, y_1) was not used in the fitting process, $MSE_1 = (y_1 - \hat{y}_1^{(1)})^2$ provides an approximately unbiased estimate for the test error. We can repeat the procedure n times and obtain the LOOCV estimate for the test MSE:

$$LOOCV = \frac{1}{n} \sum_{i=1}^n MSE_i = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i^{(i)})^2.$$

LOOCV provides approximately unbiased estimates of the test error, since each training set contains $n - 1$ observations, which is almost as many as the number of observations in the full data set. LOOCV has the potential to be very time consuming if n is very large, and if each individual model is slow to fit.

With least squares linear or polynomial regression, a shortcut makes the cost of LOOCV the same as that of a single model fit (Generalized Cross-Validation):

$$LOOCV = \frac{1}{n} \sum_{i=1}^n \left(\frac{y_i - \hat{y}_i}{1 - h_i} \right)^2$$

where $\hat{y}_i$ is the i th fitted value from the original least squares fit, and h_i is the leverage. In multiple regression, it is the i th diagonal element of the hat matrix $H = X(X'X)^{-1}X'$. This leverage lies between $1/n$ and 1, and reflects the amount that an observation influences its own fit. Note that this formula does not hold in general.

Let's apply cross validation to the `Auto` data.

The `cv.glm` function does cross-validation for glm models. It comes from the `boot` library. We use the `glm` function to fit the model. Because we don't specify a link function, it fits a linear model. We use `glm` for this because `cv.glm` only works with `glm` objects.

```
library(boot)
?cv.glm
```

The argument `K` specifies the the number of groups into which the data is divided. If `K` is not specified, then it is set to the number of observations. This results in leave-one-out CV.

```
library(boot)
glm.fit=glm(mpg~horsepower,data=Auto)
cv.err=cv.glm(Auto,glm.fit)
cv.err$delta #original cv error and bias-corrected one
```

```
## [1] 24.23151 24.23114
```

This function produces two outputs. From the help page for `cv.glm`:

A vector of length two. The first component is the raw cross-validation estimate of prediction error. The second component is the adjusted cross-validation estimate. The adjustment is designed to compensate for the bias introduced by not using leave-one-out cross-validation.

Next, we repeat this eight times with increasingly complex fits (as specified by `poly`):

```
cv.error=rep(0,8)
for (i in 1:8){
  glm.fit=glm(mpg~poly(horsepower,i),data=Auto)
  cv.error[i]=cv.glm(Auto,glm.fit)$delta[1] }
cv.error
```

```
## [1] 24.23151 19.24821 19.33498 19.42443 19.03321 18.97864 18.83305 18.96115
```

All of the higher degree polynomial models do better than linear. As we saw above, the lowest error occurs for degree 7, but the difference is small.

From this, we conclude that degree 7 is best, but that there is not a large difference as long as we use at least quadratic. Based on this, we might take our final model as degree 7, fitted with all of the data:

```
final.fit=glm(mpg~poly(horsepower,7),data=Auto)
summary(final.fit)
```

```
##
## Call:
## glm(formula = mpg ~ poly(horsepower, 7), data = Auto)
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)      23.446      0.217 108.058 < 2e-16 ***
## poly(horsepower, 7)1 -120.138      4.296 -27.966 < 2e-16 ***
## poly(horsepower, 7)2  44.090      4.296  10.263 < 2e-16 ***
## poly(horsepower, 7)3  -3.949      4.296  -0.919 0.35856
## poly(horsepower, 7)4  -5.188      4.296  -1.208 0.22794
## poly(horsepower, 7)5  13.272      4.296   3.089 0.00215 **
## poly(horsepower, 7)6  -8.546      4.296  -1.989 0.04737 *
## poly(horsepower, 7)7   7.981      4.296   1.858 0.06397 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for gaussian family taken to be 18.4548)
##
##      Null deviance: 23819.0  on 391  degrees of freedom
## Residual deviance:  7086.6  on 384  degrees of freedom
## AIC: 2265.2
##
## Number of Fisher Scoring iterations: 2
```

Exercise 2

Fit the above data with polynomials of degree 1 to 20 and use LOOCV to test the error. Which degree of polynomial produces the lowest CV error. What conclusion do you reach from this?

K-fold CV

An alternative to LOOCV is K-fold CV. This approach involves randomly dividing the set of observations into K groups, or folds, of approximately equal size. The first fold is treated as a validation set, and the method is fit on the remaining $K-1$ folds. The MSE_1 is then computed on the observations in the held-out fold. This procedure is repeated K times; each time, a different group of observations is treated as a validation set. This process results in K estimates of the test error, and the K-fold CV error estimate is computed by averaging these values:

$$\text{K-fold CV} = \frac{1}{K} \sum_{i=1}^K MSE_i.$$

LOOCV is a special case of K-fold CV in which $K = n$. In practice, one typically performs this using $K = 5$ or $K = 10$. K-fold CV is computationally less expensive than LOOCV.

The question of whether LOOCV or K-fold CV will give better estimates of the error is complicated. Performing K-fold CV will lead to an intermediate level of bias, since each training set contains $(K-1)n/K$ observations - fewer than in the LOOCV approach. Hence, from the perspective of bias reduction, it is clear that LOOCV is to be preferred to K-fold CV. However, in some circumstances LOOCV has higher variance because when we perform LOOCV, we are in effect averaging the outputs of n fitted models, each of which is trained on an almost identical set of observations. Therefore, these outputs are highly positively correlated with each other. Since the mean of many highly correlated quantities has higher variance, the test error estimate resulting from LOOCV sometimes has higher variance than does the test error estimate from K-fold CV. However, this is dependent on the particular situation and LOOCV sometimes has lower variance. See the link below for a discussion:

<https://stats.stackexchange.com/questions/61783/bias-and-variance-in-leave-one-out-vs-k-fold-cross-validation>

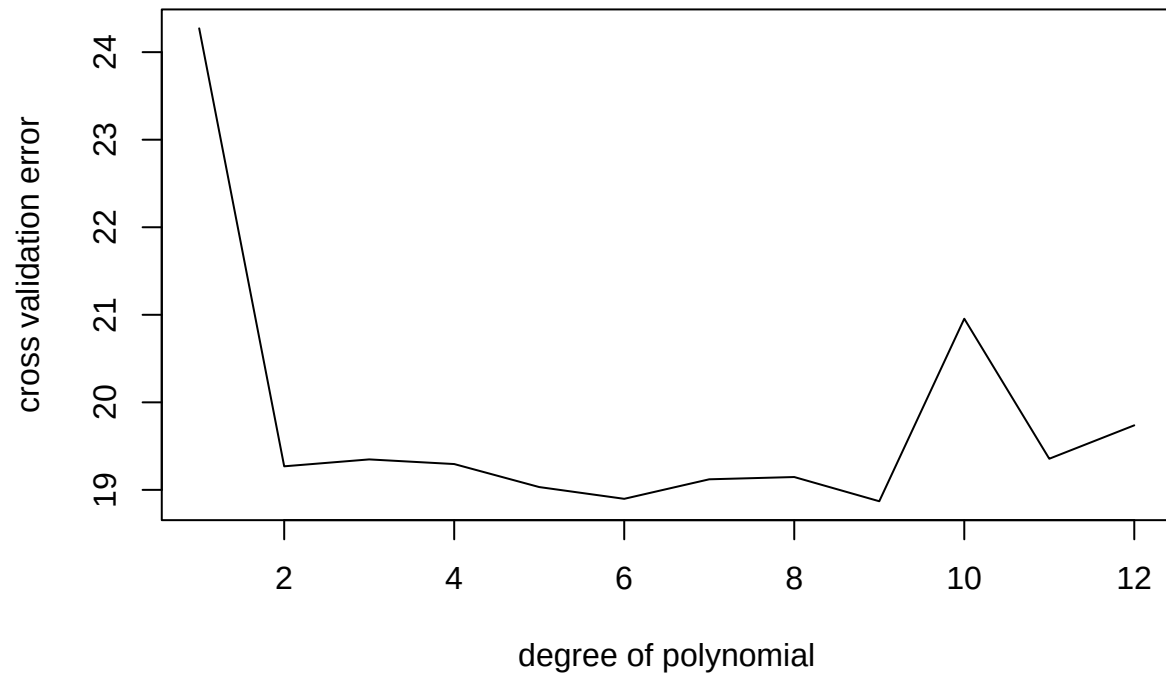
I apply 10-fold cross validation below:

```
set.seed(17)
maxfit=12
cv.error.10=rep(0,maxfit)
for (i in 1:maxfit){
  glm.fit=glm(mpg~poly(horsepower,i),data=Auto)
  cv.error.10[i]=cv.glm(Auto,glm.fit,K=10)$delta[1] }
cv.error.10
```

```
## [1] 24.27207 19.26909 19.34805 19.29496 19.03198 18.89781 19.12061 19.14666
## [9] 18.87013 20.95520 19.35554 19.73741
```

```
plot(1:maxfit,cv.error.10,type="l",xlab="degree of polynomial",ylab="cross validation error",main="10-f
```


10-fold cross validation



We will do a lot more with cross-validation as we move into regression models in the coming weeks.

Exercise 3

Conduct 5-fold cross-validation with the above data. What do you conclude about the best model?

Bias-Variance Decomposition

Next, we will look more deeply at the prediction error. Suppose that we have Y and X data related by

$$Y = f(X) + \epsilon$$

with $E(\epsilon) = 0$ and $var(\epsilon) = \sigma_\epsilon^2$.

Suppose that we fit a model $\hat{f}(x)$. The expected prediction error at an input point x_0 is given by

$$PredErr(x_0) = E[(Y - \hat{f}(x_0))^2 | X = X_0]$$

With some algebraic manipulation, this can be written as

$$PredErr(x_0) = E[(Y - f(x_0))^2] + \left[E(\hat{f}(x_0) - f(x_0))\right]^2 + E\left[\left(\hat{f}(x_0) - E(\hat{f}(x_0))\right)^2\right]$$

or

$$PredErr(x_0) = \sigma_\epsilon^2 + Bias^2 + Variance$$

The first term is the random variation of Y around its expected value when $X = x_0$. This is the intrinsic variation of the system and it cannot be avoided even with large sample size and a perfect model.

The second term is the bias of the model - that is, the expected squared difference between the fitted model and the true value of $f(x_0)$.

The third term is the variance of the fit. That is, the expected squared variance of the model fit around its mean.

Simulation of the Bias-Variance Decomposition

Next, we will simulate the components of the prediction error. The program will be completed in class.

#Completed in class

Generalization Error

Take $L(Y, \hat{f}(X))$ as a loss function for the difference between Y and its predicted value from the fitted model $\hat{f}(x)$. For a continuous response, we typically take $L(Y, \hat{f}(X))$ as the squared error.

The **test error** or **generalization error** is defined as

$$Err_\tau = E_{X^0, Y^0}(L(Y^0, \hat{f}(X^0)) | \tau)$$

τ represents the training data set. Thus, Err_τ is the expected value of the loss for a fixed training data set τ . The notation E_{X^0, Y^0} means that the expectation is over X^0 and Y^0 , which are new data values drawn randomly from their joint distribution. They are not from the training sample. Thus, this is the expected error when we apply the trained model $\hat{f}(X)$ to a new data set.

The **expected prediction error** Err averages Err_τ over the training data sets:

$$Err = E(L(Y^0, \hat{f}(X^0))) = E(Err_\tau)$$

The **training error** is defined as

$$\overline{err} = \sum_{i=1}^n L(y_i, \hat{f}(x_i))$$

To understand the differences between these different errors, focus on what is fixed and what is varying in each one:

1. **test error:** The training data set is fixed and the expectation is over the new data that is being predicted.
2. **expected prediction error:** The expectation is over the training data and the new data. The training data is no longer fixed.
3. **training error** The training data is fixed and the data being predicted is fixed (since it is the training data). There is nothing unknown.

The **test error** Err_τ is what we are typically most interested in. That is, we have a trained model and we want to know how reliable it will be with future data. Unfortunately, most methods actually estimate Err - that is, with the expectation over both the training data and the data to be predicted.

Optimism of the Training Error Rate The training error is what we have readily available because we can calculate it directly from the training sample. Unfortunately, it is not a good estimate of the test error because the same data is being used to train and test the model.

The test data will potentially vary from the training data in both the Y and the X values. It is easier to understand the difference between test and training error if we focus on the error that we would get if the values of X were the same. We call this the **in-sample error**.

$$Err_{in} = \frac{1}{N} \sum_{i=1}^N E_{Y^0} [L(Y^0, \hat{f}(x_i))]$$

In Err_{in} , we are observing new values of Y , denoted by Y_0 at the same values of X as in the training data. The notation E_{Y^0} denotes an expectation over values of Y_0 . Err_{in} is typically not the error that we are directly interested in because future data will probably not have the same X 's. However, in comparing models only the relative error matters and Err_{in} is effective for relative error.

Next, we define the **optimism** of the training error:

$$op = Err_{in} - E_Y(\overline{err})$$

This is the difference between the expected training error (training error is what we can observe directly) and the in-sample error (in-sample error is the error that we want to estimate). The training error is less than the in-sample error because we are measuring the error on the same data with which the model was trained.

The average optimism is the expectation over training data sets:

$$\omega = E_\tau(op)$$

It can be shown that for squared error and other common loss functions the average optimism is given by

$$\omega = \frac{2}{N} \sum_{i=1}^N Cov(\hat{y}_i, y_i)$$

$Cov(\hat{y}_i, y_i)$ is determined by how much the fit $\hat{y}_i$ of y_i is affected by the value of y_i .

Thus, we have

$$Err_{in} = E_Y(\overline{err}) + \frac{2}{N} \sum_{i=1}^N Cov(\hat{y}_i, y_i)$$

In the case of a linear model with d parameters and residual variance σ_ϵ^2 , we have

$$\sum_{i=1}^N Cov(\hat{y}_i, y_i) = d\sigma_\epsilon^2$$

Then,

$$Err_{in} = E_Y(\overline{err}) + \frac{2}{N} d\sigma_\epsilon^2$$

You will explore these different definitions of error via simulations in the next homework assignment.

Formula-based Estimates of In-Sample Prediction Error: C_p , AIC, BIC

So far, we have discussed estimation of the prediction error using sampling techniques. The above discussion suggest another approach:

If we can estimate the optimism ω , then we can add this to the training error to get an estimate of in-sample error

This idea is the basis for the C_p , AIC, and BIC statistics.

C_p

The general form of in-sample estimates is

$$\widehat{Err}_{in} = \overline{err} + \hat{\omega}$$

In the case of a linear model with d parameters and residual variance σ_ϵ^2 this becomes

$$\widehat{Err}_{in} = \overline{err} + \frac{2}{N} d\hat{\sigma}_\epsilon^2$$

This is a form of the well known Mallows C_p statistic:

$$C_p = \overline{err} + \frac{2}{N} d\hat{\sigma}_\epsilon^2$$

The C_p statistic is more often expressed in a different form, but the one above is equivalent.

Akaike Information Criterion (AIC)

The Akaike information criterion (AIC) is similar to C_p but is more broadly applicable. It uses the log-likelihood as a loss function. The $\widehat{Err}_{in}$ equation has the form

$$-2E[\log P_{\hat{\theta}}(Y)] \approx -\frac{2}{N} E[\log lik] + 2\frac{d}{N}$$

$P_{\hat{\theta}}(Y)$ is a density function for Y evaluated at the maximum likelihood estimate $\hat{\theta}$ for the parameters θ . The expression $loglik$ is the maximized log-likelihood

$$loglik = \sum_{i=1}^N \log P_{\hat{\theta}}(Y)$$

The left-hand side (LHS) is equivalent to $\widehat{Err_{in}}$. It is the expected value of the fitted model for future data. The first term on the right-hand side is $\overline{err}$.

For the Gaussian model, AIC is equivalent to C_p . For binomial logistic regression, it becomes

$$AIC = -\frac{2}{N} * loglik + 2 \frac{d}{N}$$

For both C_p and AIC, the model chosen is the one with the lowest value.

More generally, for a set of models with a tuning parameter α , we have

$$AIC(\alpha) = \overline{err}(\alpha) + 2 \frac{d(\alpha)}{N} \hat{\sigma}_{\epsilon}$$

AIC is an estimate of the in sample estimate. We choose the value of α that minimizes it.

Bayesian Information criterion (BIC)

BIC has a form very similar to AIC:

$$BIC = -2loglik + (\log N)d$$

Under the Gaussian model, we have

$$-2loglik = \frac{\sum_i (y_i - \hat{f}(x_i))^2}{\sigma_{\epsilon}^2} = N \frac{\overline{err}}{\sigma_{\epsilon}^2}$$

Thus,

$$BIC = \frac{N}{\sigma_{\epsilon}^2} [\overline{err} + (\log N) \frac{d}{N} \sigma_{\epsilon}^2]$$

Thus BIC is proportional to AIC, but with a factor $\log N$ replacing the factor 2.

BIC is derived in a quite different way than AIC. See the book.

We will discuss AIC and BIC more in the context of specific methods.